

torch.compile kernel fusion

in the baseline eager execution, it has a fundamental flaw:

It has no global view of the compute graph, it cannot see that `Conv → BN → ReLU` could be collapsed into a single operation.

every one of those operations in eager mode is a **seperate kernel launch**, with a separate round trip to VRAM. `torch.compile` attacks exactly this.

what kernel fusion means at the hardware level

in reager mode, a single encoder block in your U-Net looks liek this to the GPU's memory system:

EAGER MODE – Conv → BN → ReLU (3 separate kernels):

Kernel 1: Conv2d

- ① Read input tensor from VRAM ← memory load
- ② Compute convolution ← compute
- ③ Write output tensor to VRAM ← memory store

↓

[VRAM round trip]

↓

Kernel 2: BatchNorm2d

- ④ Read Conv output from VRAM ← memory load (same data!)
- ⑤ Compute normalization ← compute
- ⑥ Write BN output to VRAM ← memory store

↓

[VRAM round trip]

↓

Kernel 3: ReLU

- ⑦ Read BN output from VRAM ← memory load (same

data again!))

- | | |
|-----------------------------|----------------|
| ⑧ Compute activation | ← compute |
| ⑨ Write ReLU output to VRAM | ← memory store |

Total VRAM round trips: 6 (3 reads + 3 writes)

Total kernel launches: 3

with kernel fusion:

FUSED MODE – Conv → BN → ReLU (1 fused kernel):

Kernel 1: ConvBNReLU (fused)

- | | |
|---|-----------------------|
| ① Read input tensor from VRAM | ← memory load (once) |
| ② Compute Conv → BN → ReLU entirely inside SM registers | ← never touches VRAM |
| ③ Write final output to VRAM | ← memory store (once) |

Total VRAM round trips: 2 (1 read + 1 write)

Total kernel launches: 1

- the intermediate tensors the CONV and BN reads, the BN output that ReLU reads, **never gets written to VRAM at all**. They live and die inside the streaming multiprocessors register file, which runs at the same speed as compute itself.

the hardware change - from compute-bound to memory-bandwidth-bound

Kernel fusion doesn't primarily speed up **compute**, it speeds up **memory access**. The U-Net's Conv → BN → ReLU blocks are memory-bandwidth-bound, meaning the

GPU's Streaming Multiprocessors are fast enough that they spend more time waiting for data from VRAM than actually computing!

RTX 5070 Ti – The Two Resources:

Compute:	~1,406 TFLOPS FP16	← very fast
VRAM Bandwidth:	~896 GB/s	← fast but finite

In eager mode, each kernel launch hits **VRAM** independently.
Your **896 GB/s** gets eaten by redundant reads and writes
of intermediate tensors nobody outside the block ever needs.

Fusion eliminates those intermediate **VRAM** accesses entirely.

how it works - 3 stages

it runs a concrete pipeline before the first training step:

```
torch.compile() pipeline:
```

Stage 1: GRAPH CAPTURE (TorchDynamo)

Python bytecode → **FX Graph** (a **static** compute graph)
The compiler now has **GLOBAL** visibility into the full
forward pass **for** the first time.
It can see: "Conv output is only consumed by **BN**.
 BN output is only consumed by ReLU.
 These can be fused."

Stage 2: GRAPH OPTIMIZATION (TorchInductor)

FX Graph → Optimized **IR**
Applies fusion rules, eliminates dead tensors,
reorders operations **for** cache locality

Stage 3: CODE GENERATION (Triton / CUDA)

Optimized IR → Compiled CUDA/Triton kernels

These are written to a kernel cache on disk.

First run pays compilation cost (~60-120 seconds).

All subsequent runs load from cache – zero overhead.

parallel time complexity change

in the baseline the time for one encoder block is:

```
T_eager = T_compute_conv + T_VRAM_write + T_VRAM_read
          + T_compute_bn  + T_VRAM_write + T_VRAM_read
          + T_compute_relu
          + (3 × kernel_launch_overhead)
```

after fusion:

```
T_fused = T_compute_conv_bn_relu ← all three, but no VRAM b
etween them
          + (1 × kernel_launch_overhead)
```

```
Savings = 4 × T_VRAM_roundtrip + 2 × kernel_launch_overhead
```

contention and synchronization overhead

there are interesting interactions involving the skip connections:

CAN be fused (no data dependency conflict):

Conv → BN → ReLU ✓ linear chain, no fan-out

ConvTranspose → BN → ReLU ✓ linear chain, no fan-out

CANNOT be fused across a skip connection:

Encoder Block output → [held in VRAM] → torch.cat() in decoder

The encoder output **MUST** be written to **VRAM**.
It has to survive across potentially 15-20ms of
bottleneck and decoder compute before **cat()** uses it.
The compiler cannot fuse across that temporal gap.

`torch.compile` eliminates kernel launch overhead within blocks but cannot eliminate the fan-in synchronization barrier at the skip connections, however it reduces the number of kernels queued between those barriers, which means the pipeline drains faster when a sync is required.